Group 53 Progress Report:

Kelsey Rough, Rishi Dhanwantari, Rhea Gokhale
{roughk,macid2,gokhaler}@mcmaster.ca

1 Introduction

The Swing Trading Strategy Analyzer tests if machine learning can help decide whether a setup is strong or weak for the moving-average pullback strategy specifically. A rule-based detector scans 4-hour bars across 76 stocks. It flags a setup when a stock is trending above the 50-bar simple moving average (SMA), then pulls back into its 20-bar exponential moving average (EMA), and then continues moving up.

The machine learning model gives each setup a probability of being successful. Every trade has a stop loss $1.5\times$ the average true range (ATR) under the entry and a profit target $3\times$ the ATR above it, with a 10-bar time limit. A success means the target is reached before the stop loss is hit and before the time limit is over. The model uses price action features and macro features influenced by the overall market. Features include news tone from GDELT and the VIX volatility index.

This problem is difficult for 3 reasons:

1. There is a lot of noise in the market, so models can be wrong more often than not.

2. Financial machine learning results can be inflated by leakage, where a feature accidentally uses information from after the trade decision.

3. News data comes from a different source than the price data, so it has to be lined up carefully in time.

Due to this, we are not aiming for a super accurate model. We are aiming for one that is correct and does not leak. The project is a success if filtering setups with the model gives better trading results than taking every setup the detector finds or than just buying and holding.

2 Related Work

Krauss et al. (2017) used neural networks, gradient-boosting trees and random forest to predict which S&P 500 stocks would do better than the median the next day, based on past returns. Gradient-boosting trees performed similar to the other two methods, and results shrank in later years and with transaction costs. Like ours, their project uses tree-based methods to screen a large group of liquid stocks for short-term trades, but our model is tested across several time windows rather than one test period.

We use a version of the triple barrier method from López de Prado (2018), where each trade is classified by whether it reaches the target, a stop-loss or a time limit. The book also explains why standard k-fold cross validation results in leakage in finance datasets, since labels from overlapping trades depend on one another, so we use walk-forward splits to train on past data only. Our project differs as we use a rule-detector and meta-labeling, where the model chooses whether to take each long trade.

Rhea will add her part today!!!

3 Dataset

Data sources: This project uses hourly OHLCV data from Yahoo Finance and the yfinance library for 76 US stocks, plus SPY and QQQ as benchmark indices. The 76 tickers are split into three groups – group 1 consists of technology stocks, group 2 is media, real-estate, and industrial businesses, and group 3 has finance, energy and healthcare stocks. We've extracted the maximum amount of data points, which includes a 665-day window with a 60-day warmup for the indicators. We also get macro data from two other sources: Yahoo's daily VIX closes and the GDELT 2.0 database, which is compiled into hourly news counts and tone using Google BigQuery.

Preprocessing: We store hourly bars locally, then resample them to two bars per trading day –

one in the morning and one in the afternoon, while dropping any bars that are still in progress. We then flag any invalid OHLCV values, and remove any price jumps of over 20% or days with only one bar, to avoid corrupting our data.

Setup detection: Each setup is a long moving average pullback, detected using a rule-based detector on the 4-hour bars. To be considered a valid setup, the following must be present:

- An uptrend, where the close and EMA20 are both above the SMA50 and the SMA50 is ascending in the last 10 bars.
- A pullback, where the recent low is within 0.5% of the EMA20 in the last 3 bars and the RSI(14) is ≤ 55.
- A bounce, where the close of the signal bar is above EMA20 and above the previous bar's high.

A trade entry is done on the next bar's opening, and there can only be one position per ticker, so new setups are ignored until the current trade is complete.

Labeling: Setups are then labeled using the triple-barrier technique (López de Prado, 2018). A stop is placed about $1.5\times$ATR below the entry, and the target $3\times$ATR above it, with a limit of about 10 bars or 5 trading days. We then label setups as wins (1) if the target is hit, and time exits and stop-losses as losses (0). Setups whose trades have not finished yet get no label and are left out of training. Each trade's return is given as a value R, which is used for backtesting. When both the stop and target are within the same 4-hour bar, the hourly bar determines the first hit, otherwise the trade loses, but there were no such cases in our sample.

Dataset properties: The detector produced 1,314 setups across the 76 tickers (74 of them had at least one setup) from January 13, 2025 through November 2, 2026. Of those, 1,306 were labeled and 8 were open. Only 21.9% of setups hit the target, 44.6% hit the stop, and 33.5% expired without meeting either, so the classes are imbalanced. Since time exits can be profitable, 46.6% of trades were profitable. The average trade returned $+0.151$R, while the median is -0.19R: most trades lose a little and a few $+2$R targets raise the average. The average trade duration is 6.8 bars, and 14.8% of all setups triggered on the afternoon bar. Group 2 is notably weaker than the other groups (Table 1). Sorted by signal time, the setups are split into 1,038 training setups and 268 test setups (June 23 through October 30, 2026).

	Group 1	**Group 2**	**Group 3**	**All**
Tickers	26	25	25	76
Setups	460	382	472	1,314
Hit target	24.6%	15.4%	24.5%	21.9%
Profitable	48.5%	42.7%	47.0%	46.6%
Avg R per trade	0.233	0.022	0.174	0.151

Table 1: Detector setups and outcomes for each ticker group.

4 Features

Each setup is one row of data with 21 features. They go straight into the tree models, with no embeddings or neural network. Stock prices vary a lot, so every price-based turns into a percentage or ratio. This is so all tickers are on the same scale.

Detector features:

- RSI
- ATR is percent of price
- how far the close price is above EMA20
- how far the close price is above SMA50
- trend strength is how far the EMA20 is above the SMA50
- if the signal came on the morning or afternoon bar

Technical features: add information the detector doesn't use:

- MACD line and histogram divided by the close price
- change in the MACD histogram over the 3 pullback bars, where positive means momentum is turning upward
- 20-bar volatility of log returns
- 5-bar vs. 20-bar volatility ratio, and above 1 means volatility is getting more intense
- signal bar volume compared to the average of the last 20 bars from the same session (the morning bar is 4 hours and the afternoon bar is 2.5 hours, so they can't be mixed)

- average relative volume during the pullback where low means a quiet dip and not heavy selling

Macro features: overall market tone when the signal fires:

- VIX: latest close, its 5-day change, and its distance from its 20-day average
- GDELT, from 72 hours before the signal bar closes:
  - average news tone
  - average Goldstein score (a scale from conflict to cooperation)
  - average tone of U.S. events
  - change in tone (last 24 hours vs. the last 48 hours)
  - news volume ratio (last 24 hours vs. the last 48 hours)

Leakage prevention:

- Each feature table saves the timestamp of the newest data it used, and a check makes sure none of them are later than the signal bar's close.
- The VIX daily close only counts as available from the next day.
- A GDELT hour only counts once it is fully over.
- We also run a truncation test. It cuts the raw data off right after random setups, rebuilds the features, and checks that values for earlier setups do not change. If they did, the feature would be using future data.
- Separate checks were run for the technical and macro features.

Feature selection. We also tested relative strength features against SPY: a 20-bar relative return, a short-term relative return, and a rank across tickers. They didn't improve the walk-forward results, so we left them out. The effect of each feature group is shown in the ablation in the Implementation section.

5 Implementation

Describe your model and implementation here. Refer to item 4. This may take around a page.

6 Results and Evaluation

Splitting: Setups are sorted by signal time. The last 20% (268 setups) is the final test set and the first 80% (1,038 setups) is used for training. A random split would let the model learn from setups that happen at the same time as the test setups, which inflates results. We also use a 3-day embargo, so training trades have to close at least 3 days before the test window starts. This means trade outcomes do not overlap with the test period.

Cross-validation: Hyperparameters are only tuned on the training set, they use expanding-window walk-forward folds.

- The training data is split into 5 chunks in time order.
- Fold k trains on chunks 1 to k and validates on chunk $k + 1$, with the same embargo.
- Settings are picked by the average PR-AUC across the folds.

To have a stronger check, a full walk-forward run retrains the model across 8 time windows over the whole history and scores each window on data it has not seen.

Classification metrics: Only about 21% of test setups are wins, which makes it inaccurate. These get reported:

- precision, which is the amount of predicted wins that actually win
- recall, which is the share of actual wins that the model predicts as wins
- PR-AUC, compared against the 21% base rate
- ROC-AUC, 0.5 means random ranking
- TSS (true skill statistic), which is recall minus the false positive rate, where 0 means no skill

Trading metrics: Trades measured in R, which is profit in units of the risk taken. A full win is $+2$R and a full stop out is -1R. We report win rate, average R per trade, Sharpe ratio and max drawdown. To compare with buy-and-hold percentage returns, each trade is assumed to risk 1% of the account.

Test set (268 setups)	XGBoost	LightGBM
ROC-AUC	0.49	0.46
PR-AUC	0.21	0.20
Precision	21.1%	18.8%
Recall	26.8%	26.8%
TSS	0.004	−0.04

Table 2: Classification result for the final test set. The base rate is 20.9% wins.

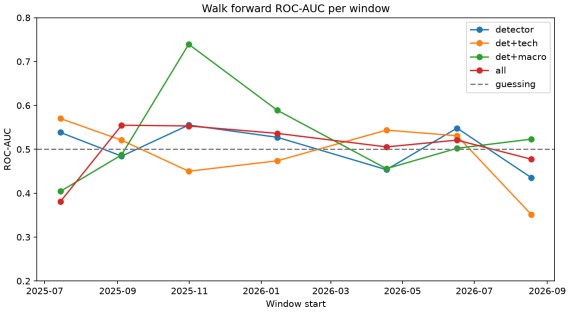


Figure 1: Walk-forward ROC-AUC for each time window and feature set, the values stay close to 0.5.

Baselines:

- buy and hold SPY and QQQ
- equal weight buy and hold of all 76 tickers
- the unfiltered detector takes every setup

The unfiltered detector is the main baseline. If the model does not beat it, the model does not add value.

Results so far: Table 2 shows the test set results. Neither model decides winners against losers better than chance, and both have precision almost equal to the base rate.

While tuning, the best settings scored about 0.30 PR-AUC, compared to fold base rates of 0.20 to 0.27. This looked like a small edge, but it disappeared on the test set. Picking the best of 24 (XGBoost) or 36 (LightGBM) settings on only about 1,000 rows gives a score that is good but only because of luck. Both models also picked the largest minimum leaf size, which is what tuning tends to do when there not much of a real pattern to find.

The walk-forward results are similar (Figure 1). Across both models and all the feature sets, ROC-AUC is between 0.49 and 0.53, and every 95% confidence interval has 0.5. ROC-AUC for individual windows ranges from 0.35 to 0.74 where there s no steady trend, which indicates luck and not a skill that will hold over time.

Table 3 shows the backtest over the approximately 90-day test window.

	Return	Sharpe	Max DD	Trades
SPY hold	+3.5%	0.91	3.5%	–
QQQ hold	−4.5%	−0.53	10.3%	–
Equal-wt hold	+9.9%	2.11	3.3%	–
Unfiltered detector	+4.1%	0.47	18.8%	268
XGBoost ($p \geq 0.5$)	+2.2%	0.43	7.8%	71

Table 3: Backtest over the final test window. Each trade risks 1% of the account. The equal weight hold is rebalanced every day.

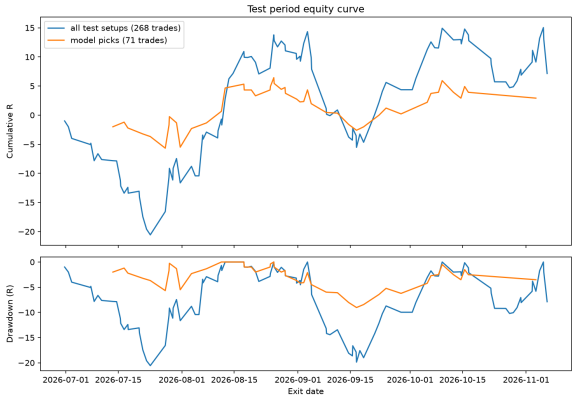


Figure 2: Cumulative R for model filtered trades compared to all of the detector setups.

- The XGBoost filter took 71 of the 268 setups.
- It returned +2.2% (Sharpe 0.43), compared to +4.1% (Sharpe 0.47) for the unfiltered detector.
- It reduced the max drawdown from 18.8% to 7.8%, mostly because it took fewer trades.
- The equal weight buy and hold returned +9.9% (Sharpe 2.11), which is higher than both.

The model's picks earned a little more per trade than all the setups on the test set, 0.041R compared to 0.027R (Figure 2). With ROC-AUC around 0.5, it is likely that the difference is due to noise. Total R was also lower for the model (2.9R vs 7.2R) because it took less trades.

Over the full history, the unfiltered detector averaged +0.157R per trade over 1,315 trades. This suggests the pullback pattern is mildly positive on its own, but our current features do not predict which setups will work.

7 Feedback and Plans

Write about your plans for the remainder of the project. This should include a discussion of the feedback you received from your TA, and how you

plan to improve your approach. Reflect on your implementation and areas for improvement. Refer to item 6. This may be around 0.5 pages.

Equations

1. Return in R. The return of each trade is measured in R, which is its profit divided by the amount risked:

$$R = \frac{\text{exit price} - \text{entry price}}{\text{entry price} - \text{stop price}}, \tag{1}$$

$$\text{stop price} = \text{entry price} - 1.5 \times \text{ATR}, \tag{2}$$

where ATR is the average true range. A stop exit is therefore about -1R and a target exit about $+2$R.

2. Purging. To prevent leakage, a training setup is kept only if its trade closed before the evaluation window starts, minus an embargo of 3 days:

$$t_{\text{exit}} < t_{\text{eval start}} - 3 \text{ days}. \tag{3}$$

3. Sharpe ratio. The Sharpe ratio is computed on daily R totals, where days with no exits count as 0, and is annualized over 252 trading days:

$$\text{Sharpe} = \frac{\overline{R}_{\text{daily}}}{\sigma_{R,\text{daily}}} \times \sqrt{252}. \tag{4}$$

4. Maximum drawdown. The maximum drawdown is the largest drop of the cumulative R curve below its previous peak. Trades that close at the same time are added together first, so the result does not depend on row order. With $C(t)$ as the cumulative R at time t:

$$\text{MDD} = \max_t \left(\max_{s \le t} C(s) - C(t) \right). \tag{5}$$

5. True skill statistic. Because wins are rare, we report the true skill statistic (TSS) instead of accuracy:

$$\text{TSS} = \frac{TP}{TP + FN} - \frac{FP}{FP + TN}. \tag{6}$$

TSS is 0 for random guessing and 1 for a perfect classifier.

Limitations

One limitation is that the Yahoo data covers only two years, which means we have a single market condition, an uptrend. The test period was also short (approximately 90 days with 268 setups), so the margin of error is big, as we have ROC-AUC 95% confidence intervals of approximately ± 0.08. To account for this, we backtested the whole history of the stocks and got about 1,000 out-of-sample predictions. We did not account for commissions, slippage, or position limits, and set the risk per setup to 1R, which is 1% of the account in the backtest. We chose 76 tickers, considering the availability of Yahoo Finance and GDELT data, which may bias the samples towards stocks that are more often covered by these sources. We also considered several combinations of features, target values, and models, which increases the risk of a multiple comparison problem, where a model may appear statistically significant even if it is not.

Team Contributions

Below is a summary of each team member's work since the proposal:

Kelsey set up the GitHub repository, implemented the OHLCV data pull and resampling to 4-hour bars, wrote the setup detector and triple barrier labeler, and wrote the tests for them. Kelsey also implemented the shared leakage detection, which was run on all of the macro features. She also trained and tuned XGBoost on a purged time split, including the full-history evaluation step, the backtest metrics such as win rate, Sharpe ratio, and maximum drawdown, and the result visualization including the dashboard for per-setup model scores.

Rishi created the GDELT and VIX macro data pipeline using BigQuery aggregation, caching, and macro features, and merged them with setups. He also wrote the macro leakage check script, and completed the LightGBM training.

Rhea set up access to the GDELT dataset through BigQuery, implemented an indicator library (including EMA, SMA, ATR, RSI, and MACD), implemented technical features such as MACD, volatility, volume and their leakage checks. She was also responsible for dataset compilation and cleaning, detector output comparison for ticker groups and definition of baseline strategies such as the buy-and-hold and the unfiltered detector.

References

Christopher Krauss, Xuan Anh Do, and Nicolas Huck. 2017. Deep neural networks, gradient-boosted trees, random forests: Statistical arbitrage on the S&P 500. *European Journal of Operational Research*, 259(2):689–702.

Marcos López de Prado. 2018. *Advances in Financial Machine Learning*. Wiley, Hoboken, NJ.